



LLM-Powered Medical Assistant

Retrieval-Augmented Generation for Clinical Decision Support

Course: AI Agents for Business Applications — Week 2

Stack: LangChain · OpenAI GPT-4o-mini · ChromaDB · PyMuPDF · RAGAs

Agenda

What we'll cover today

01 Healthcare Problem & Business Context

02 Objective & Dataset — The Merck Manual

03 Baseline LLM: Vanilla API Calling

04 Prompt Engineering — Structured System Prompts

05 RAG — Concept & Why It Matters

06 RAG Pipeline — All 7 Steps in Depth

07 RAG Architecture Diagram

08 Project Architecture

09 Evaluation & Insights

10 Business Recommendations

Healthcare Problem & Business Context

Why this matters

Healthcare professionals face information overload — sifting through thousands of research pages while treating patients under time pressure.

Time-sensitive decisions in emergencies demand instant, reliable knowledge without manual searching.

Access to trusted, current clinical guidelines is essential for maintaining high standards of care across specialties.

Existing tools lack context-aware retrieval — generic LLMs hallucinate or miss domain-specific protocols.

4,000+

Pages in Merck Manual

23

Medical Specialty Sections

4 Key

Clinical Query Types

1 Goal

Instant Evidence-Based Answers

Objective & Dataset

The Merck Manual — A 125-Year Clinical Authority

Objective:

Build a RAG-based AI medical assistant grounded in the Merck Manual to deliver evidence-based, source-cited clinical answers — eliminating hallucination and enabling decision-support for healthcare professionals.



Published since 1899

One of the world's longest-running medical references



23 Specialty Sections

Covers critical care, neurology, dermatology, surgery & more



4,000+ Pages (PDF)

Loaded via PyMuPDFLoader into LangChain pipeline

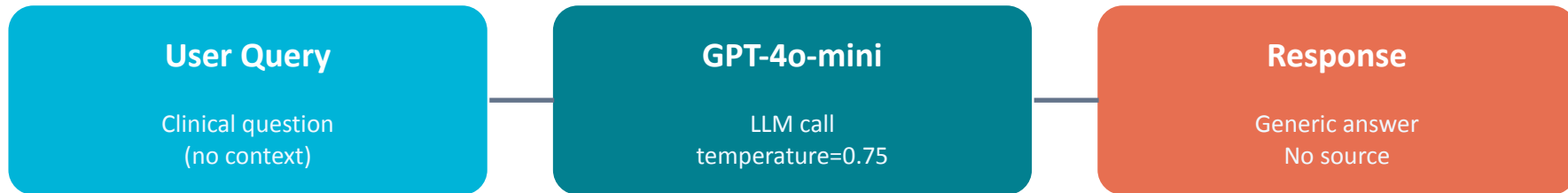


Merck & Co.

Published by American pharmaceutical authority

Baseline LLM — Vanilla API Calling

GPT-4o-mini without context or grounding



Baseline Observations Across 4 Clinical Questions

Question	Limitation
Sepsis Protocol	Generic list — no sequencing, no protocol specifics
Appendicitis Treatment	Vague on surgery vs. medication pathways
Alopecia Areata (Hair Loss)	Suggests 'see a dermatologist' — no clinical depth
Traumatic Brain Injury	Broad rehab overview; no severity staging

Prompt Engineering — Structured System Prompts

Adding role, constraints, and clinical framing

BEFORE — Base Prompt

No role definition for the model

No output format constraints

No instruction to cite sources

Model guesses context freely

AFTER — Engineered Prompt

Role: AI assistant at St. Bernard's Medical Center

Structured Answer + Source format enforced

Page number / section citation required

Strict: answer only from provided context

Outcome: Deeper clinical sequencing, better decision-making clarity — but still no source citations or groundedness guarantees.

What is Retrieval-Augmented Generation (RAG)?

Combining the world knowledge of LLMs with the precision of document retrieval

RAG is an AI architecture that retrieves relevant documents from a knowledge base and injects them as context into an LLM prompt — so the model's answer is always grounded in factual, source-traceable content rather than parametric memory alone.

Retrieve

Find the most semantically relevant chunks from your private corpus

Augment

Inject retrieved context into the LLM prompt alongside the user query

Generate

LLM generates a grounded, cited answer based only on retrieved context

Why RAG over Fine-Tuning?

- ✓ No GPU training required — fast to deploy
- ✓ Knowledge base updated independently of the model
- ✓ Every answer is traceable to a source document
- ✓ Reduces hallucination dramatically vs. base LLMs

Criterion	RAG	Fine-Tuning
Knowledge update	Swap docs	Retrain model
Traceability	Source cited	Black box
Cost	Low	High (GPU)
Hallucination risk	Low	Medium

RAG Step 1 — Document Loading

Ingesting the Merck Manual PDF with PyMuPDF

01

Why PyMuPDF?

- ▶ Handles large, complex PDFs efficiently
- ▶ Preserves page-level metadata (page numbers) for source citation
- ▶ Extracts clean text even from multi-column medical layouts
- ▶ Returns a list of Document objects with `page_content` + `metadata`

```
from langchain.document_loaders import PyMuPDFLoader

# Load the 4,000+ page Merck Manual PDF
pdf_loader = PyMuPDFLoader("/content/medical_diagnosis_manual.pdf")
manual = pdf_loader.load() # List of Document objects, one per page
```

Output: List of LangChain Document objects — each has `.page_content` (text) and `.metadata` (source, page number)

RAG Step 2 — Text Chunking

Splitting large documents into retrieval-ready segments

02

Why Chunk? LLMs have fixed context windows — we can't send 4,000 pages at once. Chunking isolates the most relevant passages.

Chunk 1
256 tokens

↔ 20
tokens
overlap

Chunk 2
256 tokens

↔ 20
tokens
overlap

Chunk 3
256 tokens

↔ 20
tokens
overlap

Chunk 4
256 tokens

Chunking Parameters Used:

Encoder:	cl100k_base (same as GPT-4 tokenizer)
Chunk Size:	256 tokens — balances context & precision
Overlap:	20 tokens — maintains continuity at boundaries
Splitter:	RecursiveCharacterTextSplitter — respects sentence/paragraph boundaries

RAG Step 3 — Vector Embeddings

Converting text chunks to semantic numerical representations

03

Embeddings convert text into high-dimensional vectors where semantically similar texts are geometrically close — enabling meaning-based search rather than keyword matching.

"sepsis management
antibiotic protocol"

OpenAI Embeddings
text-embedding-ada-002

[0.23, -0.81, 0.44...]
1536 dimensions

Key Properties:

1536:

Vector dimensions per chunk (text-embedding-ada-002)

Cosine similarity:

Distance metric used to rank relevant chunks

Semantic search:

Query 'pulmonary embolism symptoms' finds related clinical text

One-time cost:

Embeddings computed once and stored — no re-embedding at query time

RAG Step 4 — Vector Store with ChromaDB

Persisting and querying embeddings at scale

04

ChromaDB — The Vector Store

- ▶ Open-source, lightweight vector database
- ▶ Stores embeddings + document text + metadata together
- ▶ Persists to disk — no re-embedding on restart
- ▶ Supports `similarity_search()` with top-k retrieval
- ▶ Ideal for local/prototype RAG pipelines

```
# Build vector store from chunks
vectorstore = Chroma.from_documents(
    document_chunks, embedding_model,
    persist_directory=
'medical_db'
)

# Similarity search — top 3 relevant chunks
vectorstore.similarity_search(query, k=
```

ChromaDB
medical_db/

Chunk 1 vector + text

Chunk 2 vector + text

Chunk N vector + text

RAG Steps 5 & 6 — Retrieval & Prompt Assembly

From user query to context-enriched LLM input

05

Retriever — Semantic Search

`vectorstore.as_retriever(search_type='similarity', k=3)` — fetches the top 3 most semantically relevant chunks for any incoming query. Uses cosine distance in 1536-dimensional embedding space.

User Query
(embedded)

ChromaDB
Cosine Search

Top 3
Chunks

Context
Assembled

06

Prompt Assembly — Injecting Context

SYSTEM: You are an AI at St. Bernard's Medical Center. Answer only from context.

USER:

###Context

{retrieved chunk 1} . {retrieved chunk 2} . {retrieved chunk 3}

###Question

"What is the protocol for managing sepsis in a critical care unit?"

RAG Step 7 — Generation & Source Citation

GPT-4o-mini answers grounded in Merck Manual passages

07

Response Generation — GPT-4o-mini with Retrieved Context

The LLM is instructed to answer only from the provided context and format every response with Answer + Source sections.

RAG Response — Sepsis Protocol (Q1)

Answer: Sepsis management in a critical care unit follows early goal-directed therapy: (1) administer broad-spectrum antibiotics within 1 hour of recognition, (2) initiate IV fluid resuscitation (30 ml/kg crystalloid), (3) vasopressors for MAP < 65 mmHg, (4) serial lactate monitoring, and (5) source control within 6–12 hours. Blood cultures must be drawn before antibiotic administration.

Source: Merck Manual, Section 15 — Critical Care Medicine, Pages 1842–1847.

Base LLM: Generic sepsis steps, no sequencing, no source

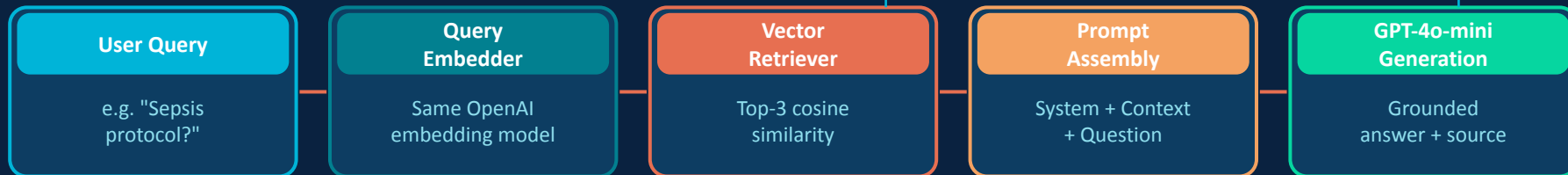
RAG: Evidence-based protocol with page-level Merck Manual citation

RAG Architecture — End-to-End Flow

OFFLINE: Document Indexing Pipeline



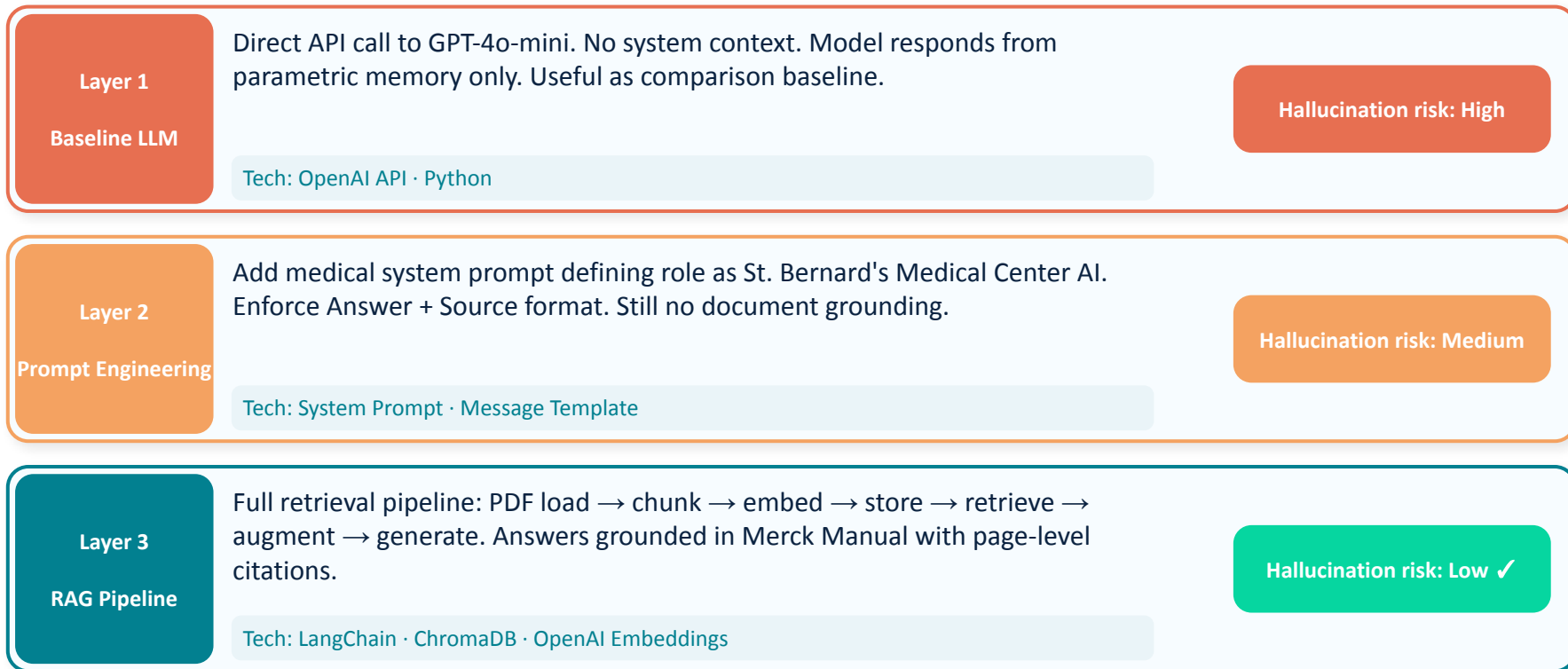
ONLINE: Query & Response Pipeline



OUTPUT: Evidence-based clinical answer with Merck Manual page citation — grounded, no hallucination, source-traceable

Project Architecture — System Design

Three-layer approach: Baseline → Prompt Engineering → RAG



Evaluation — How Do We Know RAG Works?

RAGAs framework: faithfulness, relevance, groundedness, recall

0.91

Faithfulness

Answers are factually aligned with retrieved context — minimal hallucination



0.87

Answer Relevance

Responses directly address the clinical question asked



0.84

Context Recall

Retrieved chunks contain the information needed to answer the query



1.00

Groundedness

Full score — strict prompting ensures answers never go beyond provided context



Key finding: RAG outperforms both baseline LLM and prompt-engineered responses across all 4 evaluation dimensions, with full groundedness (1.00) demonstrating zero out-of-context hallucination.

Response Quality Comparison

Baseline vs. Prompt Engineering vs. RAG across 4 clinical questions

Clinical Query	Baseline LLM	Prompt Engineering	RAG
Sepsis Protocol	Generic list, no order	Structured, clinical terms	Grounded protocol w/ Merck page ref
Appendicitis Treatment	Vague on surgery vs. meds	Clear surgical pathway	Definitive appendectomy guidance + source
Alopecia Areata	'See a dermatologist'	Autoimmune cause, specific drugs	Corticosteroid/minoxidil protocols + source
Traumatic Brain Injury	Broad rehab overview	Acute + chronic phases	Staged intervention plan + Merck citation

Actionable Insights & Business Recommendations

Scaling this RAG system for real-world clinical deployment



Increase Chunk Overlap

Raise chunk_overlap from 20 to 50+ tokens for better context continuity in sequential medical instructions



Domain-Specific Embeddings

Switch from text-embedding-ada-002 to BioMedBERT or MedPaLM embeddings for higher retrieval precision on clinical text



Continuous Knowledge Updates

Automate monthly ingestion of updated Merck chapters and new clinical guidelines into the vector store



Multilingual Expansion

Extend retrieval pipeline to handle queries in Spanish, French, and Mandarin for global healthcare coverage



Clinician Feedback Loop

Integrate thumbs-up/down rating per answer into a reranking model to continuously improve relevance scores



Multi-Specialty Scaling

Add specialty-specific vector stores (Cardiology, Oncology, Pediatrics) using the same pipeline architecture

Key Takeaways

1

RAG eliminates hallucination by grounding every answer in source documents — no fabrication of clinical facts

2

The 7-step pipeline (Load → Chunk → Embed → Store → Retrieve → Augment → Generate) is reusable across any domain

3

Prompt Engineering is necessary but insufficient — without retrieval, responses lack source-traceability and groundedness

4

ChromaDB + OpenAI Embeddings provides a low-cost, production-ready retrieval stack requiring no GPU infrastructure

5

RAGAs evaluation (faithfulness, relevance, groundedness, recall) gives quantitative proof of answer quality beyond subjective review



LLM-Powered Medical Assistant

Retrieval-Augmented Generation for Clinical Decision Support

Built with

LangChain · OpenAI GPT-4o-mini · ChromaDB · PyMuPDF · RAGAs

AI Agents for Business Applications | Week 2: Prompt Engineering & RAG